

CSA0506-DBMS CO4 AT3 ANSWERS	BY: Antony Magrin (192572103)
--	---

Q1. An e-commerce company needs to store 10 million product records.
Recommend a file organization strategy and justify your choice with cost analysis.

Requirement Analysis

An e-commerce catalog is read-heavy: products are searched by ProductID, category, and price range far more often than they are inserted or deleted. Customers also expect fast filtering (range queries on price, rating) and fast exact lookups (ProductID, SKU).

Recommended Strategy

- **Primary organization: Indexed Sequential (ISAM-like) or a B+ Tree clustered index on ProductID, since it gives both fast direct access and efficient range scans.**
- **Secondary indexes: B+ Tree non-clustered indexes on Category, Price, and Rating to support filtering/sorting used in product listing pages.**
- **Hashing (optional): A hash index can be added purely for exact-match SKU/barcode lookups at checkout, where $O(1)$ access matters most.**

Cost Analysis (approximate)

Assume each product record is 200 bytes and the block size is 4 KB (4096 bytes).

1. Blocking factor = $4096 / 200 \approx 20$ records per block.
2. Number of blocks for 10,000,000 records $\approx 10,000,000 / 20 = 500,000$ blocks.
3. Heap file exact-match search (worst case): up to 500,000 block accesses (full scan) — unacceptable for a live storefront.
4. B+ Tree search (fan-out ≈ 100 per node): height $\approx \log_{100}(500,000) \approx 3$ levels $\rightarrow$ about 3–4 disk I/Os per lookup, i.e., roughly 125,000 times fewer I/Os than a heap scan.

5. Extra storage cost of indexes: a B+ Tree index on a 4-byte key plus 4-byte pointer needs roughly 8 bytes per entry; for 10 million records this is about 80 MB per index — negligible compared to the multi-GB product data, and well worth the retrieval speed-up.

Justification

Given the scale (10 million rows) and the read-heavy, filter-heavy workload typical of an e-commerce catalog, a B+ Tree-based indexed organization (clustered on ProductID with secondary B+ Tree indexes on Category/Price) offers the best balance: logarithmic search time, efficient range filtering for browsing/sorting, and only modest extra storage overhead. Pure heap storage would be cheapest to maintain but is unacceptable for live query latency; pure hashing would speed up exact lookups but cannot support the range and sort queries essential to a shopping experience.

Q2. A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.

Query Pattern

- **Exact-match lookups:** "Get all transactions for AccountNumber = X" (point/equality query).
- **Range queries:** "Get transactions for AccountNumber = X between Date1 and Date2" (range query on a secondary attribute).

Proposed Design

1. Primary clustered index on AccountNumber: Since transactions are almost always retrieved per account, physically clustering (sorting) the Transactions table by AccountNumber (and TransactionDate as a secondary sort key within each account) means all of a customer's transactions sit in contiguous blocks — a single scan retrieves them.
2. Composite B+ Tree index on (AccountNumber, TransactionDate): building the index on the combined key allows the DBMS to first navigate to the correct account, then use the sorted date ordering within that branch to satisfy the date-range condition efficiently without scanning unrelated accounts.
3. Secondary non-clustered index on TransactionDate alone: useful for bank-wide reports (e.g., "all transactions across all accounts on a given day") that are not filtered by account.
4. Hashing is avoided as the primary structure here because hashing destroys the ordering needed for the date-range portion of the query, even though it would speed up pure account-number equality lookups.

Why This Works

The composite (AccountNumber, TransactionDate) B+ Tree index matches the exact query shape used by the application (equality on account, range on date), which is the standard best-practice rule for composite index column ordering: equality columns first, range columns last. This gives $O(\log n)$ access to the start of the range, followed by a fast sequential scan through the linked B+ Tree leaves for the date range, avoiding any full table scan.

Q3. A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.

Query Pattern

- **PatientID lookups:** front-desk/registration systems fetch a specific patient's full record by a unique PatientID.
- **Doctor name lookups:** doctors or admin staff pull up "all patients assigned to Dr. X", a non-unique, non-ordering field.

Multi-Level Indexing Design

1. Level 0 (data file): Patient records stored in a data file, physically clustered/sorted by PatientID (the primary key).
2. Level 1 (primary index): A sparse first-level index on PatientID — one entry per data block, since the file is already sorted on this key. This first-level index is itself treated as a file.
3. Level 2 (second-level index): Because even the first-level index can be large for millions of patients, a second, smaller sparse index is built over the first-level index (an index of an index), forming a two-level tree so the topmost index fits in memory. This is the standard multi-level indexing / B+ Tree principle.
4. Secondary dense index on DoctorName: since DoctorName is not the file's ordering key and many patients can share the same doctor, a separate dense, multi-level (B+ Tree) secondary index is built with DoctorName as the search key, with each index entry pointing to the corresponding patient records (using a bucket/pointer list where a doctor has many patients).

Resulting Access Paths

Query	Path Used	Approx. Cost
Find patient by PatientID	Top-level index → first-level index → data block	$O(\log n)$, typically 2–3 I/Os
Find all patients under Dr. X	Secondary multi-level index on DoctorName → list of matching record pointers	$O(\log n)$ to locate + $O(k)$ to fetch k matching patients

This two-index, multi-level design keeps the primary structure efficient for the common unique-key access (PatientID) while still supporting fast, scalable lookups on the non-key DoctorName attribute without forcing a full table scan.

Q4. A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.

Nature of the Workload

Shipment records are created continuously (new shipments booked), updated as status changes, and eventually archived/deleted once delivered — a highly dynamic, write-heavy workload, alongside operational queries such as "show all shipments dispatched between two dates" or "show current status of shipment ID X".

Analysis

Criterion	B-Tree	B+ Tree
Insertion/deletion cost	Rebalancing (splits/merges) can occur at any level since data pointers exist in internal nodes too, making updates to internal nodes more expensive.	Rebalancing is generally cheaper and more localized because only leaf nodes hold data; internal nodes just guide the search and change less often.

Criterion	B-Tree	B+ Tree
Range/sequential queries (e.g., shipments in a date range, tracking history)	Slower — no linked-leaf structure, so a range scan requires repeated tree traversals.	Fast — leaf nodes are linked in sequence, so range scans just walk the linked list after one initial descent.
Predictability of operation cost	Less predictable, since data may be found at varying depths.	Uniform, since every search always ends at a leaf — easier to reason about worst-case performance under heavy concurrent updates.
Storage efficiency for internal nodes	Internal nodes are larger since they also store data pointers, reducing fan-out.	Internal nodes store only keys, increasing fan-out and reducing tree height for the same data volume.

Conclusion

For a logistics system with frequent, ongoing insertions and deletions of shipment records plus a strong need for date-range and status-range queries (tracking, reporting), the B+ Tree is more suitable. Its higher fan-out reduces tree height (fewer I/Os per update), its leaf-linking makes range queries efficient, and localizing data only to leaves keeps insert/delete rebalancing cheaper and more predictable than a classic B-Tree.

Q5. Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.

Design

Assume each employee record is 100 bytes and disk block size is 4 KB (4096 bytes), giving a blocking factor of about 40 records per block.

1. Choose EmployeeID as the hash key (unique, fixed-length, uniformly distributed if IDs are sequential — apply a hash function such as $h(\text{EmployeeID}) = \text{EmployeeID} \bmod N$ to avoid clustering from sequential IDs).
2. With 5000 employees and 40 records/block, minimum blocks needed $\approx 5000/40 = 125$ blocks. Allowing a load factor of about 80% for room to grow, allocate $N \approx 160$ buckets.
3. Each bucket is allocated one disk block; overflow chaining is used for any bucket that exceeds capacity due to collisions.

Static Hashing vs. Extendible Hashing for This Scenario

Aspect	Static Hashing	Extendible Hashing
Bucket count	Fixed at 160 buckets, decided upfront.	Starts small and grows via a directory as the employee count increases.
Suitability for 5000 (relatively stable) employees	Well suited — HR headcount grows slowly and predictably, so a fixed bucket count sized with headroom performs very well with minimal overhead.	Overkill for this scale — the extra directory-management complexity is not justified when growth is slow and bounded.
Overflow handling	Minor overflow chaining possible if hiring exceeds projections, but manageable with the 20% headroom.	Handles unpredictable growth more gracefully by splitting only the overflowing bucket.
Implementation cost	Simple, low overhead.	Higher overhead — requires maintaining and possibly doubling a directory structure.

Recommendation

For an HR system of this size (5000 employees, slow and predictable growth), static hashing sized with roughly 20% headroom (about 160 buckets) is the more cost-effective and simpler choice. Extendible hashing would only be justified if the company expected rapid, unpredictable headcount growth (e.g., mass hiring/layoffs) where a fixed bucket count would quickly become a bottleneck.

Q6. An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.

Query Pattern

- **Point query:** "Show seat availability for Flight AI202" (exact-match on FlightNumber).
- **Range query:** "Show all flights departing between 10 Sep and 15 Sep" (range on DepartureDate, possibly combined with FlightNumber or Route).

Proposed Combined Structure

1. Primary hash index on FlightNumber: since flight number lookups are simple equality checks that occur extremely frequently (every booking/check-in), a hash index gives near $O(1)$ access to a specific flight's record.
2. Secondary B+ Tree index on DepartureDate: since dates need ordered, range-based retrieval, a B+ Tree (with linked leaf nodes) is layered on top so that a date range can be scanned efficiently once the starting date is located.
3. Composite B+ Tree index on (Route, DepartureDate) for queries like "flights from Chennai to Delhi in the next week", following the equality-then-range column ordering rule.

Why a Combined (Hybrid) Structure

No single index type optimally serves both access patterns: hashing is ideal for the flight-number point query but cannot support ordered range scans on date, while a B+ Tree supports both point and range queries but is marginally slower than hashing for pure equality lookups. Combining a hash index for FlightNumber with a B+ Tree for DepartureDate (and composite Route+Date) gives each query type its best-suited structure, at the modest cost of maintaining two index structures instead of one.

Q7. For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.

Query Pattern

- **"List all students in the Computer Science department"** (equality on Department).
- **"List all students with CGPA between 8.0 and 9.0"** (range on CGPA), often combined with department (e.g., top CS students).

Design

1. Primary file organization: store the Students data file clustered/sorted by StudentID (the primary key), since it is the most common unique-identifier lookup (e.g., transcripts, ID card verification) and supports a primary sparse index.
2. Secondary index on Department: a dense secondary B+ Tree (or simple secondary) index, since Department is a non-ordering, low-cardinality field — this allows quick retrieval of all students in a given department without scanning the whole file.
3. Secondary index on CGPA: a dense B+ Tree secondary index on CGPA supports efficient range queries (e.g., CGPA between 8.0 and 9.0) because B+ Tree leaves are linked in sorted order.
4. Composite secondary index on (Department, CGPA): built specifically to serve combined queries such as "CS students with CGPA above 8.5," following the equality-column-first, range-column-last ordering rule — this avoids the DBMS having to intersect two separate index results.

Summary Table

Index	Type	Purpose
Primary index on StudentID	Sparse, clustered	Fast lookup of an individual student's full record
Secondary index on Department	Dense, non-clustered	Fast retrieval of all students in a department
Secondary index on CGPA	Dense, B+ Tree	Efficient CGPA range queries
Composite index on (Department, CGPA)	Dense, B+ Tree, composite	Efficient combined department + CGPA-range queries

Q8. Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.

Base Figures

Block size = 4 KB = 4096 bytes. Record size = 50 bytes. Maximum records that can physically fit per block = $\lfloor 4096 / 50 \rfloor = 81$ records/block ($81 \times 50 = 4050$ bytes used, 46 bytes unused per fully packed block).

1. Heap File Organization

Heap files pack records tightly in arrival order with no reserved free space, so blocks can be filled to (near) their physical maximum.

- Records per block ≈ 81 (maximum packing).
- Total blocks required = $\lceil 100,000 / 81 \rceil = 1,235$ blocks.
- Block utilization $\approx 4050 / 4096 \approx 98.9\%$ (very high, since no space is deliberately left free).

2. Sorted (Sequential) File Organization

Sorted files conventionally reserve free space in each block (a common assumption is a 70% packing/fill factor) so that new records can be inserted in the correct sorted position without immediately triggering a block split or overflow area.

- Effective records per block $\approx 0.70 \times 81 \approx 56$ records/block.
- Total blocks required = $\lceil 100,000 / 56 \rceil = 1,786$ blocks.
- Block utilization $\approx 70\%$ by design (deliberately lower to leave room for insertions), meaning roughly 45% more blocks are needed than the heap file for the same data.

3. Hashed File Organization

Hashed files are typically sized to a target load factor of about 80% to keep collision/overflow chains short while still leaving room for new inserts.

- Effective records per block $\approx 0.80 \times 81 \approx 64$ records/block.
- Total blocks required (buckets) = $\lceil 100,000 / 64 \rceil = 1,563$ blocks, plus a small reserve of overflow blocks (commonly an additional $\sim 5\text{--}10\%$) for buckets exceeding capacity due to collisions.
- Block utilization $\approx 80\%$ by design, giving a middle ground between the heap file and sorted file.

Comparative Summary

Organization	Fill Factor	Blocks Needed	Utilization
Heap File	$\sim 100\%$ (tightly packed)	1,235	$\approx 98.9\%$
Sorted File	$\sim 70\%$ (reserved for insertions)	1,786	$\approx 70\%$

Organization	Fill Factor	Blocks Needed	Utilization
Hashed File	~80% (target load factor)	1,563 (+ overflow reserve)	≈ 80%

Interpretation

The heap file achieves the highest space (block) utilization because it has no ordering constraint to preserve and simply packs records as they arrive. The sorted file uses the most blocks (lowest utilization) because it must deliberately reserve free space in every block to accommodate future insertions without immediately violating sort order. The hashed file sits in between, striking a balance between space efficiency and keeping collision/overflow chains short. This illustrates the general trade-off analyzed in Q1/Q8-style problems: organizations optimized for insertion flexibility and ordered access sacrifice some raw storage efficiency compared to a simple heap.

Q9. A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.

Query Pattern

- "Show all posts by user_id = X, most recent first" (equality on user + range/sort on timestamp).
- "Show recent posts under #hashtag" (equality on hashtag + range on timestamp) — a many-to-many relationship since one post can have multiple hashtags.

Design at This Scale

At 500 million rows, no single flat index is practical; the design must combine partitioning, composite indexing, and specialized structures.

1. Partitioning (sharding) by user_id range or hash: split the posts table across many physical partitions/shards keyed on user_id, so that a query for one user only touches one shard rather than the full 500-million-row table — this is essential at this scale before indexing even comes into play.
2. Composite B+ Tree index on (user_id, timestamp DESC) within each partition: supports the very common "user's own timeline" query — equality on user_id first, then range/sort on timestamp, matching the standard composite-index ordering rule and allowing chronological scans directly from the leaf-linked B+ Tree.

3. Inverted index on hashtag: since a post can carry many hashtags and a hashtag maps to many posts (many-to-many), use an inverted-index structure (hashtag → sorted list/postings of post IDs with timestamps), similar to search-engine indexing, rather than a plain B+ Tree — this is the standard approach for high-cardinality tag-style lookups at scale.
4. Composite index on (hashtag, timestamp) within the inverted index postings list: keeps each hashtag's post list sorted by recency, so "trending/recent posts for #hashtag" can be served directly without re-sorting.
5. Caching/hot-tier hashing: an additional hash-based cache (e.g., an in-memory hash index) for the most recently active users/hashtags reduces load on the B+ Tree/disk structures for the hottest queries, common in real-world large-scale systems.

Why Not a Single Index

A single composite index cannot efficiently serve all three access patterns (by user, by time, by hashtag) simultaneously because each has a different natural key ordering and different cardinality characteristics. Partitioning reduces the effective index size per query, the (user_id, timestamp) B+ Tree serves the timeline use case, and the hashtag inverted index serves the tag-search use case — together forming a composite, workload-driven indexing strategy appropriate for 500 million records.

Q10. Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.

Context

A supermarket billing system is dominated by insertions (every checkout adds a new transaction record continuously through the day), occasional exact-match lookups (e.g., "pull up receipt/transaction ID X" for a return or audit), and end-of-day range queries/reports (e.g., "all transactions between 6 PM and 9 PM"), with essentially no in-place deletions during the day (deletions, if any, only happen in bulk during archival).

Performance Comparison

Operation	Heap File	Sorted File	Hashed File
Insertion	Very fast — $O(1)$; new transaction simply appended at the end of the file as it's billed.	Slow — $O(n)$ in the worst case; the record must be placed in correct sorted (e.g., by TransactionID/time) position, potentially shifting records or writing to an overflow area.	Fast — $O(1)$ average; hash function computes the bucket directly, though occasional collisions add minor overhead.
Deletion	Fast — $O(1)$ to mark as deleted (tombstone); no reordering needed. Suits this system since deletions are rare/batched.	Moderate — $O(n)$ worst case if physical reorganization is needed; $O(1)$ if just tombstone-marked, but periodic reorganization is still required.	Fast — $O(1)$ average to remove/mark the record in its bucket.
Search (exact match, e.g., TransactionID)	Slow — $O(n)$; requires a full linear scan since there's no order or hash structure.	Fast — $O(\log n)$ via binary search, since the file is sorted.	Very fast — $O(1)$ average; direct bucket computation from the key.
Search (range, e.g., transactions in a time window)	Very slow — $O(n)$; must scan the entire file and filter.	Fast — $O(\log n)$ to find the start, then a fast sequential scan through the range.	Very slow — $O(n)$; hashing scatters records with no ordering, so effectively the whole file must be scanned.

Recommendation for This Scenario

Given that the supermarket billing system's dominant operation is high-volume, continuous insertion (1 million transactions/day) with only occasional exact lookups and end-of-day range reporting, a heap file

is best for the live, in-progress transaction log during the day, since its $O(1)$ insertion keeps checkout throughput high and its weak search performance is tolerable at low query frequency. For the exact-match "look up a specific receipt" need, a supplementary hash index over TransactionID can be layered on top of the heap file without changing the insertion path. At day's end, the day's heap file can be batch-loaded into a sorted (or B+ Tree indexed) archive to support fast historical range queries and reporting — combining the strengths of each organization at the stage of the data lifecycle where it matters most.